

Mobile Robotics Lab Manual

Lab Session 4: Introduction to ROS 2 Launch, Rosbag, rqt-plot

Azmeer Ali

University of Engineering and Technology (UET) Lahore

May 10, 2026

1 Objective

The primary objectives of this lab were to understand and utilize the ROS 2 launch system, record and replay ROS 2 topics using `rosvbag`, and visualize topic data utilizing `rqt_plot`. Specifically, a follow-the-leader behavior was implemented involving two `turtlesim` nodes.

2 Methodology and Approach

2.1 Follow-the-Leader Node Implementation

A custom ROS 2 node, `turtle_follower`, was created using Python. The node subscribes to the pose of the leading turtle (`/turtle1/pose`) and the following turtle (`/turtle2/pose`). It calculates the Euclidean distance and the angular difference between the two turtles. Using a proportional controller, it publishes velocity commands (`Twist` messages) to `/turtle2/cmd_vel` to minimize this error. A threshold distance of 0.5 units was set to prevent the following turtle from jittering upon reaching the target.

2.2 ROS 2 Launch File Generation

A python-based launch file, `turtlesim_launch.py`, was created to execute multiple nodes simultaneously. It was designed to start the main `turtlesim_node`, delay the spawning of the second turtle to avoid race conditions using `TimerAction`, execute the `turtle_follower` node, and open a separate terminal for `turtle_teleop_key`.

2.3 Data Recording and Visualization

The velocity topics for both turtles (`/turtle1/cmd_vel` and `/turtle2/cmd_vel`) were recorded using the `rosvbag2` utility. Following data collection, `rqt_plot` was used to visualize the linear and angular velocity commands being published to `turtle1` in real-time.

3 Implementation Code

3.1 Follower Logic (Python Snippet)

```
1 def control_loop(self):
2     msg = Twist()
3     distance = math.sqrt((self.target_pose.x - self.current_pose.x)**2
4     +
5     (self.target_pose.y - self.current_pose.y)**2)
6
7     if distance > 0.5:
8         msg.linear.x = 1.2 * distance
9         angle_to_target = math.atan2(self.target_pose.y - self.
10         current_pose.y,
11         self.target_pose.x - self.
12         current_pose.x)
13         angle_error = angle_to_target - self.current_pose.theta
14
15         # Normalize angle to [-pi, pi]
16         while angle_error > math.pi:
17             angle_error -= 2.0 * math.pi
18         while angle_error < -math.pi:
19             angle_error += 2.0 * math.pi
20
21         msg.angular.z = 4.0 * angle_error
22     else:
23         msg.linear.x = 0.0
24         msg.angular.z = 0.0
25
26     self.publisher_.publish(msg)
```

Listing 1: Turtle Follower Control Loop

4 Findings and Observations

- **Proportional Control Response:** The proportional controllers successfully maneuvered `turtle2` towards `turtle1`. Tuning the gain values (1.2 for linear and 4.0 for angular velocity) proved critical; higher values led to overshoot, while lower values caused lagging.
- **Launch File Synchronization:** Incorporating a `TimerAction` in the launch file resolved an initial race condition where the service call to spawn `turtle2` was executed before the main `turtlesim` node had initialized.
- **Trajectory Analysis:** Extracting data from the `rosvim` file showed corresponding peaks in `turtle2`'s velocity slightly delayed from `turtle1`'s movements, confirming the expected tracking delay inherent in the publish/subscribe architecture and the physical distance gap.

5 Conclusion

The lab effectively demonstrated the modularity of ROS 2. By combining launch files, bag recording, plotting tools, and custom control loops, a complete robotic system was

established and visualized. The follow-the-leader application served as a practical introduction to basic kinematic control and ROS 2 topic interactions.